

GL Capture / Replay Developer Overview

Updated: 2026-09-28 15:32:35

This document summarizes current capture/replay architecture and the latest reliability/UX updates.

Architecture

- 1) gl_capture_ui launches target suspended, injects gl_capture_hook.dll, then resumes.
- 2) gl_capture_hook intercepts OpenGL/WGL calls and writes trace records (GLCAP001 + per-call payload).
- 3) gl_replay decodes trace and replays against a fresh OpenGL context with ID remapping.
- 4) Replay UI shows call list, state panels, attachment previews, and run-to-call navigation.

Recent Stability Fixes

- Added custom capture/replay for glGetBooleanv, glGetFloatv, glGetIntegerv.
- Fixed query result-size mappings that caused heap corruption in pglview.exe.
 - * GL_MAX_VIEWPORT_DIMS (0x0D3A) now uses 2 elements.
 - * GL_POINT_SIZE_RANGE (0x0B12) now uses 2 elements.
 - * GL_LINE_WIDTH_RANGE (0x0B22) now uses 2 elements.
- Replay query decoder hardened for short/malformed payloads.

Pointer Argument Visibility in Call List

- Pointer-captured custom calls now include original pointer value (ptr=0x...) in trace payload.
- Call list now displays decoded typed values for fixed-size vector/matrix pointer APIs.
- glLightfv and glLightModelfv call-list formatting now shows light/pname + ptr + values.
- Query APIs show pname, pointer, count, and returned values preview.
- Note: old traces captured before this change may not contain pointer addresses.

Current Limitations

- glInterleavedArrays currently records format/stride/pointer-presence metadata only.
- Full arbitrary pointer API expansion in UI depends on captured payload semantics per function.
- Non-custom generic pointer wrappers still serialize presence-only metadata for safety.

Build and Run

Build: cmake --build . --config Debug --target gl_capture_hook gl_replay gl_capture_ui --parallel 4

Launcher: gl_capture_ui -e <exe> -i "<args>" -d <run_dir> -o <trace_path>

Replay: set GLCAP_REPLAY_AUTOEXIT=1 && gl_replay <trace_file>